

Sesión 6: Seguimiento de experimentos con MLflow Tracking

Duración: 2,5 horas | Modalidad: Virtual síncrona

Objetivo	Dejar de usar hojas de cálculo o nombres de archivo para rastrear experimentos. Adoptar MLflow Tracking como sistema centralizado y reproducible.
Herramientas	Python, MLflow, scikit-learn, Hugging Face (transformers, datasets)
Prerrequisitos	Sesiones 1-5 completadas. Python 3.10+, pip install mlflow scikit-learn transformers datasets torch

BLOQUE 1 — El caos de la experimentación (20 min)

Empezamos con una pregunta directa al grupo: ¿cómo lleváis el registro de vuestros experimentos? La respuesta habitual incluye carpetas con nombres como `modelo_v3_final_DEFINITIVO.pkl`, hojas de cálculo desactualizadas o, simplemente, nada. Ese es el problema que MLflow resuelve.

Los síntomas más comunes del caos experimental son:

- Modelos guardados como `model_rf_lr0.01_depth5_v2_real.pkl`
- Métricas apuntadas en un Excel que nadie actualiza después del primer día
- Imposibilidad de reproducir el 'mejor modelo' de hace dos semanas
- Equipos donde cada miembro tiene su propia convención de nombres

Punto clave

Este no es un problema de organización personal. Es un problema estructural que ocurre en todos los equipos sin herramientas adecuadas. MLflow impone estructura de forma automática.

BLOQUE 2 — Introducción a MLflow (20 min)

MLflow es una plataforma open source para gestionar el ciclo de vida del machine learning. Tiene cuatro componentes, pero en esta sesión nos centramos en uno: Tracking.

Los conceptos clave que hay que interiorizar:

- Experiment: agrupador lógico de ejecuciones (ej. 'clasificador-sentimientos-v1')
- Run: una ejecución concreta con sus parámetros, métricas y artefactos registrados
- Artifact: cualquier fichero que quieras conservar (el modelo, gráficas, CSVs de resultados...)
- MLflow Tracking Server: proceso que recibe y almacena toda esa información

Instalación del entorno para esta sesión:

```
pip install mlflow scikit-learn transformers datasets torch evaluate

# Arrancar el servidor de MLflow (ejecutar en terminal separada)
mlflow ui
# Abre http://localhost:5000 en el navegador
```

BLOQUE 3 — El experimento: clasificación de sentimientos con HuggingFace (40 min)

Vamos a usar un modelo preentrenado de Hugging Face para clasificación de sentimientos (DistilBERT fine-tuneado en SST-2). El objetivo es comparar varias configuraciones de un pipeline de predicción y encontrar la mejor mediante MLflow, sin entrenar un modelo desde cero.

¿Por qué DistilBERT?

DistilBERT es una versión ligera de BERT con el 97% de su rendimiento y la mitad de parámetros. El modelo `distilbert-base-uncased-finetuned-sst-2-english` está disponible directamente en HuggingFace Hub y no requiere GPU para inferencia.

Fichero: `experiment_tracking.py`

```
import mlflow
import mlflow.sklearn
import numpy as np
import json
from transformers import pipeline
from datasets import load_dataset
from sklearn.metrics import accuracy_score, f1_score, classification_report
import time

# — CONFIGURACIÓN —
EXPERIMENT_NAME = 'sentiment-analysis-hf'
mlflow.set_experiment(EXPERIMENT_NAME)

# Cargamos el dataset de test de SST-2 (solo las primeras N muestras para
agilizar)
print('Cargando dataset SST-2...')
dataset = load_dataset('glue', 'sst2', split='validation[:200]')
texts = dataset['sentence']
```

```

labels = dataset['label'] # 0=negativo, 1=positivo

# — CONFIGURACIONES A COMPARAR —————
# Variamos: modelo base, tamaño del batch y umbral de confianza
configs = [
    {
        'model_name': 'distilbert-base-uncased-finetuned-sst-2-english',
        'batch_size': 16,
        'confidence_threshold': 0.5,
        'truncation': True,
        'max_length': 128
    },
    {
        'model_name': 'distilbert-base-uncased-finetuned-sst-2-english',
        'batch_size': 32,
        'confidence_threshold': 0.7, # Umbral más alto = más conservador
        'truncation': True,
        'max_length': 128
    },
    {
        'model_name': 'distilbert-base-uncased-finetuned-sst-2-english',
        'batch_size': 16,
        'confidence_threshold': 0.9, # Muy conservador
        'truncation': True,
        'max_length': 64
    }
]

# — FUNCIÓN DE EVALUACIÓN —————
def run_experiment(config):
    with mlflow.start_run():
        # 1. Registrar TODOS los parámetros
        mlflow.log_params({
            'model_name': config['model_name'],
            'batch_size': config['batch_size'],
            'confidence_threshold': config['confidence_threshold'],
            'max_length': config['max_length'],
            'dataset': 'sst2-validation-200',
            'truncation': config['truncation']
        })

        # 2. Cargar el pipeline de HuggingFace
        print(f"Cargando modelo {config['model_name']}...")
        clf = pipeline(
            'sentiment-analysis',
            model=config['model_name'],
            truncation=config['truncation'],
            max_length=config['max_length']
        )

        # 3. Inferencia y medición de latencia
        start = time.time()
        raw_preds = clf(list(texts), batch_size=config['batch_size'])
        latency = time.time() - start

```

```

# 4. Aplicar umbral de confianza
# Si la confianza es < threshold, marcamos como 'incierto' (se descarta
del eval)
preds_filtered, labels_filtered = [], []
uncertain_count = 0
label_map = {'POSITIVE': 1, 'NEGATIVE': 0}

for pred, true_label in zip(raw_preds, labels):
    if pred['score'] >= config['confidence_threshold']:
        preds_filtered.append(label_map[pred['label']])
        labels_filtered.append(true_label)
    else:
        uncertain_count += 1

# 5. Calcular métricas
if len(preds_filtered) == 0:
    print(' WARN: threshold demasiado alto, 0 predicciones conservadas.')
    return

acc = accuracy_score(labels_filtered, preds_filtered)
f1 = f1_score(labels_filtered, preds_filtered, average='weighted')
coverage = len(preds_filtered) / len(labels)
avg_latency_ms = (latency / len(texts)) * 1000

# 6. Registrar MÉTRICAS en MLflow
mlflow.log_metrics({
    'accuracy': round(acc, 4),
    'f1_weighted': round(f1, 4),
    'coverage': round(coverage, 4),
    'uncertain_samples': uncertain_count,
    'avg_latency_ms': round(avg_latency_ms, 2),
    'total_latency_s': round(latency, 2)
})

# 7. Registrar ARTEFACTOS
# 7a. Reporte de clasificación como fichero de texto
report = classification_report(
    labels_filtered, preds_filtered,
    target_names=['Negativo', 'Positivo']
)
with open('/tmp/classification_report.txt', 'w') as f:
    f.write(f'Config: {json.dumps(config, indent=2)}\n\n')
    f.write(report)
mlflow.log_artifact('/tmp/classification_report.txt')

# 7b. Predicciones con scores guardadas como JSON
predictions_detail = [
    {'text': t, 'true': l, 'pred': p, 'score': r['score']}
    for t, l, p, r in zip(texts, labels, preds_filtered +
['N/A']*uncertain_count, raw_preds)
]
with open('/tmp/predictions.json', 'w') as f:
    json.dump(predictions_detail[:20], f, ensure_ascii=False, indent=2) #
Primeras 20
mlflow.log_artifact('/tmp/predictions.json')

```

```

        print(f' acc={acc:.4f} | f1={f1:.4f} | coverage={coverage:.2%} |
latencia={avg_latency_ms:.1f}ms')
        return acc

# — EJECUTAR TODAS LAS CONFIGURACIONES —————
if __name__ == '__main__':
    print(f'Lanzando {len(configs)} experimentos...')
    for i, cfg in enumerate(configs):
        print(f'\n[{i+1}/{len(configs)}] threshold={cfg["confidence_threshold"]}
batch={cfg["batch_size"]}')
        run_experiment(cfg)
    print('\nExperimentos completados. Abre http://localhost:5000 para comparar
resultados.')

```

BLOQUE 4 — La UI de MLflow (20 min)

Una vez ejecutado el script, arrancamos la UI y la exploramos en vivo. Los puntos clave que demostrar:

- Navegar al experimento 'sentiment-analysis-hf' y ver las 3 runs
- Seleccionar las 3 runs y pulsar 'Compare' para verlas en paralelo
- Filtrar por accuracy > 0.90 para encontrar la mejor configuración
- Abrir una run específica y descargar los artefactos (classification_report.txt)
- Ver el apartado 'Params' vs 'Metrics' y entender la diferencia

```

# Comando para abrir la UI (si no está abierta ya)
mlflow ui --port 5000

# Para buscar la mejor run programáticamente (útil en scripts de CD)
import mlflow

client = mlflow.MlflowClient()
experiment = client.get_experiment_by_name('sentiment-analysis-hf')

best_run = client.search_runs(
    experiment_ids=[experiment.experiment_id],
    order_by=['metrics.accuracy DESC'],
    max_results=1
)[0]

print(f'Mejor run ID: {best_run.info.run_id}')
print(f'Accuracy: {best_run.data.metrics["accuracy"]}')
print(f'Params: {best_run.data.params}')

```

TRABAJO AUTÓNOMO (2,5 h)

Tarea para casa

Ampliar el script añadiendo al menos dos configuraciones nuevas. Probar con un modelo diferente de HuggingFace (por ejemplo 'cardiffnlp/twitter-roberta-base-sentiment-latest') y comparar los resultados en la UI de MLflow. Llegar a la sesión 7 con al menos 5 runs registradas.

Lecturas recomendadas:

- MLflow Tracking documentation: <https://mlflow.org/docs/latest/tracking.html>
- 'Towards MLOps: Technical capabilities of a machine learning platform' (IEEE Access)